SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
NAME:	**V.SUSHMITHA**	**HALLTICEKT:**	**2303A54055**
TIME	**MON**	**Applicable to Batches**	**47_B**
Assignment Number: 9.1			

	Lab Experiment: Documentation Generation -Automatic documentation and code comments **Lab Objectives** 1. To understand automatic documentation generation. 2. To generate code comments and docstrings using AI tools. 3. To learn the importance of documentation in software development.	
1	**Lab Outcomes** 1. Students will be able to generate documentation automatically for code. 2. Students will be able to add clear comments and docstrings to programs. 3. Students will be able to improve code readability and maintainability using documentation.	Week5 - Monday
	Problem 1: Consider the following Python function: def find_max(numbers):	

```
return max(numbers)
```

Task:

- Write documentation for the function in all three formats:

(a) Doc string : In this format, the documentation is written inside triple quotes just below the function definition.The doc string explains that:

- The function takes a list of numbers as input.
- It finds the maximum value in the list.
- It returns the largest number.

This format keeps the explanation simple and clear. It is placed inside the function, so anyone reading the code can immediately understand what the function does.

(b) Inline comments : In this format, comments are written inside the function using the # symbol.The inline comments explain:

- That the function receives a list of numbers.
- That it uses Python's built-in method to find the maximum value.
- That it returns the result.

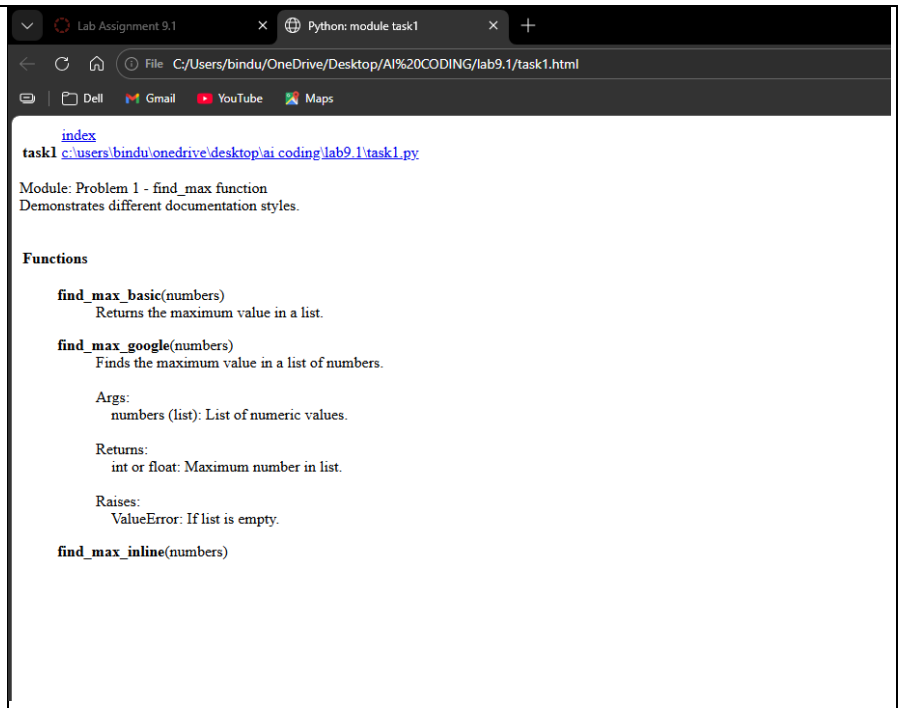
Inline comments mainly explain the internal working of the function step by step. They are helpful when the logic is complex.

(c) Google-style documentation : In this format, the documentation is written in a structured way inside the docstring.A short description of what the function does.The arguments it takes (the list of numbers).This style is more detailed and organized. It is mostly used in professional projects.

Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style. :
DOCSTRING

- Simple and easy to write.
- Automatically used by documentation tools.
- Good for small programs.

	Inline Comments • Helpful to explain logic clearly. • Good for understanding complex code. • But cannot be used to generate automatic documentation. • Not suitable as official documentation. Google-Style Documentation • Very clear and well-structured. • Explains arguments and return values properly. • Suitable for large projects and libraries. • Takes more time to write. • **Recommend which documentation style is most effective for a mathematical utilities library and justify your answer :** For a mathematical utilities library, Google-style documentation is the most effective.This is because mathematical functions must clearly mention: • What type of input they expect. • What output they return. • What errors may occur. Since such libraries are reused by many developers, proper and structured documentation is very important. **PROMPT USED :** **Python3 –m pydoc task01** **Python3 –m pydoc –w task01** **start task01.html** **output :**	



Problem 2: Consider the following Python function:

```
def login(user, password, credentials):
return credentials.get(user) == password
```

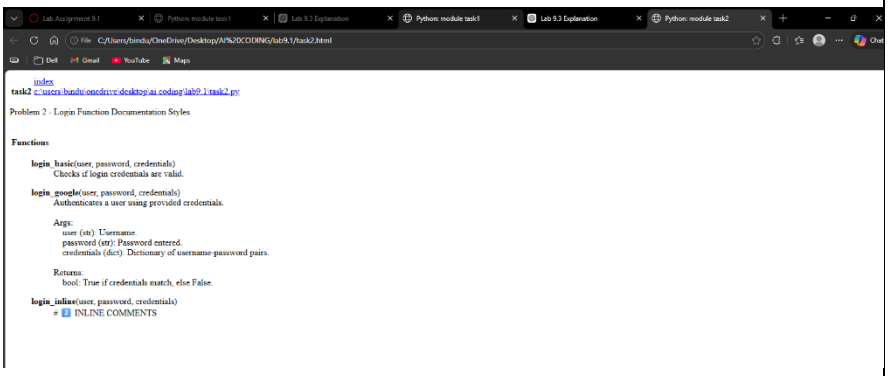
Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

Prompt used : python -m pydoc task2

python -m pydoc -w task2

Start task02.html



Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.

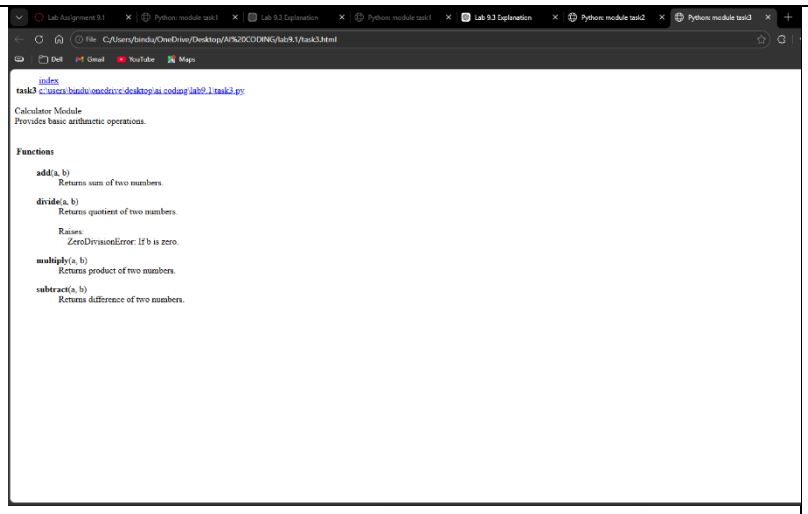
Instructions:

1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:
   - add(a, b) – returns the sum of two numbers
   - subtract(a, b) – returns the difference of two numbers
   - multiply(a, b) – returns the product of two numbers
   - divide(a, b) – returns the quotient of two numbers
2. Display the module documentation in the terminal using Python's documentation tools.
3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

   Prompt used : python -m pydoc task3

   python -m pydoc -w task3

   Start task03.html



Problem 4: Conversion Utilities Module

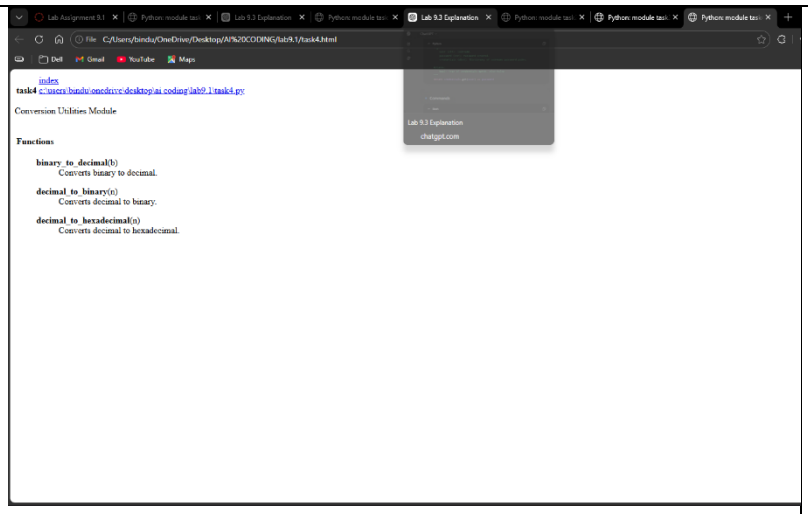
Task:

1. Write a module named conversion.py with functions:
   - decimal_to_binary(n)
   - binary_to_decimal(b)
   - decimal_to_hexadecimal(n)
2. Use Copilot for auto-generating docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

   Prompt used : python -m pydoc task4

   python -m pydoc -w task4

   Start task04.html



Problem 5 – Course Management Module

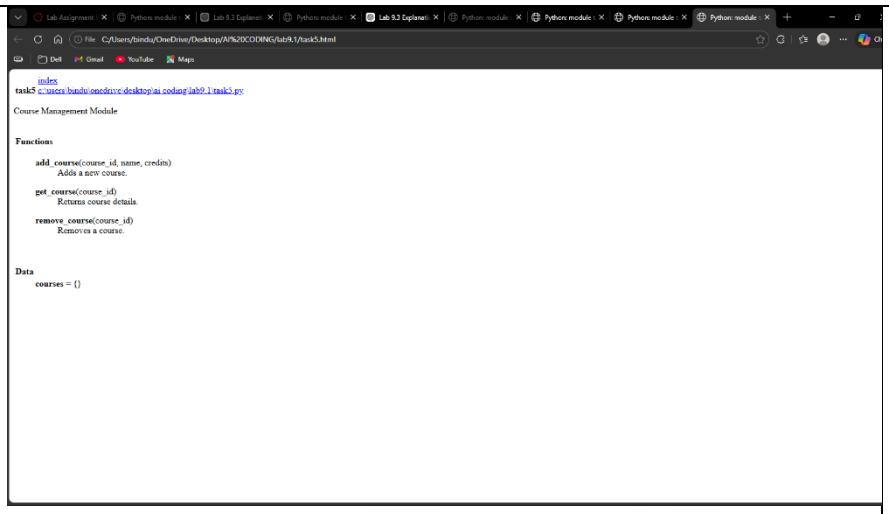
Task:

1. Create a module course.py with functions:
   - add_course(course_id, name, credits)
   - remove_course(course_id)
   - get_course(course_id)
2. Add docstrings with Copilot.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

Prompt used : python -m pydoc task5

python -m pydoc -w task5

Start task05.html


Lab Assignment
Python: module
Lab 9.3 Explanati
Python: module
Lab 9.3 Explanati
Python: module
Python: module
Python: module
Python: module
File C:/Users/bindu/OneDrive/Desktop/AI%20CODING/lab9.1/task5.html
Dell
Gmail
YouTube
Maps
index
task5 c:\users\bindu\onedrive\desktop\ai coding\lab9.1\task5.py
Course Management Module
Functions
add_course(course_id, name, credits)
Adds a new course.
get_course(course_id)
Returns course details.
remove_course(course_id)
Removes a course.
Data
courses = {}